Justificación del Sistema de Información y Control del Transmetro: Propósito, Fundamento y Problemática que Resuelve

Luis Orozco

Carné 3590 22 5339

Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas

Aseguramiento de la Calidad del Software

2 de octubre de 2026

Justificación del Sistema de Información y Control del Transmetro

El Sistema de Información y Control del Transmetro (SIC Transmetro) tiene como propósito centralizar en una sola base de datos la información de 7 líneas, 106 estaciones y 210 unidades del sistema de transporte de la Ciudad de Guatemala y municipios aledaños, así como automatizar las reglas de operación que actualmente dependen de la verificación manual. El presente documento expone, para el sistema en conjunto y para cada decisión de arquitectura, tres aspectos: para qué sirve, por qué se adoptó frente a las alternativas evaluadas y qué problema concreto resuelve.

Planteamiento del Problema

La información del Transmetro se encuentra distribuida entre cada municipalidad y cada operador, sin que exista una fuente oficial única. Las reglas de operación están documentadas en papel y se verifican mediante observación directa del personal.

Tabla 1

Situaciones identificadas en la operación actual y sus consecuencias

Situación actual	**Consecuencia**
Catálogos en archivos separados por municipalidad	El mismo dato existe en varias versiones y ninguna es oficial
Asignación de unidades registrada en hojas de cálculo	Una unidad puede quedar contabilizada en dos líneas o en ninguna
Control del rango de unidades por observación	La falta de unidades se detecta cuando el usuario ya espera en la estación
Parqueos asignados de forma verbal	Unidades que terminan la jornada sin espacio asignado
Cobertura de guardias en listados impresos	Accesos descubiertos que no se detectan oportunamente
Conteo de pasajeros sin registro sistemático	El sobrecupo no deja evidencia y no puede ser reclamado
Regla de espera de cinco minutos a criterio del operador	Se cumple de forma discrecional
Conciliación del recaudo con varios días de diferencia	Los faltantes se descubren cuando ya no es posible rastrearlos
Ausencia de registro de cambios	Ante un error no existe forma de determinar responsabilidad ni de reconstruir el hecho

Nota. Elaboración propia a partir del levantamiento de requerimientos del proyecto.

Propósito del Sistema

Tabla 2

Propósitos del sistema y usuarios destinatarios

Propósito	**Usuario destinatario**
Mantener un catálogo único y oficial de líneas, estaciones, accesos y unidades	Administrador municipal y administrador general
Conocer en todo momento cuántas unidades opera cada línea y si se encuentra dentro de su rango	Administrador municipal y administrador general
Garantizar que ninguna unidad esté asignada a dos líneas ni permanezca sin parqueo	Administrador municipal
Administrar el expediente de pilotos y la cobertura de guardias por acceso	Administrador municipal, piloto y guardia
Registrar la ocupación real de cada unidad y detectar el sobrecupo	Operador de estación y piloto
Hacer obligatoria la regla de espera de cinco minutos	Operador de estación
Conocer el estado del equipo de cómputo y la conectividad de cada estación	Operador de estación
Conciliar el recaudo por tarjeta el mismo día de la operación	Operador de estación
Obtener reportes consolidados de la red, exportables a formatos estándar	Administrador general
Determinar quién realizó cada cambio, cuándo y sobre qué valores	Auditoría y administración

Nota. Los usuarios corresponden a los cinco roles definidos en la matriz de permisos del sistema.

Fundamento de las Decisiones de Arquitectura

Cada decisión se expone bajo el mismo esquema: finalidad, fundamento frente a la alternativa descartada y problema que resuelve.

Base de Datos Relacional Centralizada

Finalidad. Disponer de un único lugar de consulta para cualquier dato de la red, con independencia de la municipalidad que lo haya capturado.

Fundamento. La alternativa evaluada consistía en mantener una base de datos por municipalidad y consolidarlas periódicamente. Ese esquema implica que, durante el intervalo entre consolidaciones, ninguna instancia posee el dato correcto, y que las diferencias deben resolverse manualmente. Se seleccionó PostgreSQL por tratarse de un motor de código abierto, requisito del proyecto, y por ofrecer las capacidades que el diseño requiere (PostgreSQL Global Development Group, s.f.).

Problema que resuelve. Elimina las versiones paralelas del mismo dato. Cada municipalidad conserva la propiedad de su información, representada mediante una columna en cada tabla, pero los datos residen en un solo repositorio.

Reglas de Negocio Implementadas en la Base de Datos

Finalidad. Asegurar que una regla se cumpla siempre, con independencia del medio por el cual ingresen los datos.

Fundamento. La práctica habitual consiste en programar las reglas en la capa de aplicación. Bajo ese esquema, la regla solo se cumple si el dato ingresó por esa vía: una carga masiva, un script de corrección o una consulta manual la omiten sin generar aviso. Al implementarla en el motor, la validación forma parte del dato y no del programa que lo escribe. Esta decisión corresponde al principio de ubicar la lógica de integridad en la capa que la puede garantizar de forma incondicional (Sommerville, 2016).

Problema que resuelve. Que una regla especificada en el documento de requerimientos se cumpla efectivamente en producción y no únicamente en el flujo previsto de la aplicación.

Tabla 3

Reglas de negocio implementadas en el motor de base de datos

Requerimiento	**Mecanismo**	**Situación que impide**
RF-04. Una unidad en una sola línea activa	Restricción EXCLUDE USING gist sobre unidad y periodo	Que la misma unidad se contabilice en dos líneas, incluso ante asignaciones simultáneas
RF-06. Ninguna unidad sin parqueo	Dos restricciones EXCLUDE	Que dos unidades ocupen el mismo espacio o que una unidad figure en dos predios
RF-09. Sobrecupo	Columnas generadas con umbral del 150 % de la capacidad	Que la detección del sobrecupo dependa de un cálculo manual
RF-10. Espera de cinco minutos	Restricción CHECK sobre el registro del viaje	Que se despache con menos del 25 % de ocupación antes del tiempo mínimo
RF-05. Rango de unidades por línea	Disparador que abre y cierra la alerta	Que una línea opere fuera de su rango sin generar aviso
RF-08. Cobertura de guardias	Disparador que compara contra el mínimo del acceso	Que un acceso permanezca descubierto sin detección

Nota. RF corresponde a requerimiento funcional. Las siete reglas fueron verificadas con datos inválidos sobre PostgreSQL 18.3.

Arquitectura en Tres Capas con Interfaz REST

Finalidad. Concentrar la lógica en un solo lugar y garantizar que todos los clientes, tanto la aplicación web actual como posibles aplicaciones móviles o tableros de análisis, obtengan resultados idénticos.

Fundamento. La alternativa era una aplicación que accediera directamente a la base de datos, lo cual obliga a repetir las validaciones en cada pantalla y a exponer el motor a cada equipo de la red, con el consiguiente riesgo de seguridad. Con una interfaz REST intermedia, la base acepta conexiones únicamente desde el servidor de aplicación.

Problema que resuelve. Evita que cada nueva pantalla reimplemente las reglas y permite modificar la interfaz sin alterar la lógica. Satisface además el requerimiento no funcional de capas desacopladas.

Interfaz Web Responsiva Basada en Componentes

Finalidad. Permitir el uso del sistema tanto desde la computadora de la estación como desde un dispositivo móvil en el andén.

Fundamento. El operador de estación y el guardia desempeñan sus funciones de pie. Una aplicación de escritorio los obligaría a regresar a la caseta para registrar lo observado, y el registro diferido pierde exactitud. Se seleccionó React por su modelo de componentes reutilizables y por la disponibilidad de documentación y soporte, factor relevante en un sistema que será mantenido por terceros.

Problema que resuelve. Que la ocupación de una unidad se registre en el momento y en el lugar en que ocurre.

Matriz de Permisos Almacenada como Dato

Finalidad. Que cada uno de los cinco roles acceda y modifique exactamente lo que le corresponde.

Fundamento. Programar los permisos mediante condiciones en el código implica que cada ajuste requiere modificación, prueba y despliegue del programa. Al almacenarlos en una tabla, el ajuste consiste en actualizar un registro.

Problema que resuelve. Que un piloto no acceda al expediente de otro, que un operador no modifique el catálogo de líneas y que un administrador municipal no altere datos de otra municipalidad. Adicionalmente, corregir un permiso mal asignado toma minutos y no un ciclo de despliegue.

Ámbito de Datos Independiente del Nivel de Permiso

Finalidad. Distinguir qué operaciones puede ejecutar un usuario de cuáles registros puede afectar.

Fundamento. Dos administradores municipales poseen el mismo nivel de permiso pero no deben acceder a los mismos datos. Combinar ambos conceptos obligaría a crear un rol por municipalidad, con un crecimiento incontrolado del catálogo de roles.

Problema que resuelve. Que el sistema escale a nuevos municipios sin rediseño: incorporar uno consiste en insertar un registro y no en crear roles adicionales.

Alertas Generadas por el Motor

Finalidad. Detectar las situaciones anómalas en el instante en que se producen.

Fundamento. La alternativa consistía en un proceso de revisión periódica, que deja una ventana sin cobertura entre ejecuciones y consume recursos consultando información que no ha cambiado. Un disparador reacciona al cambio mismo.

Problema que resuelve. Que la alerta por una línea bajo su mínimo exista en el mismo instante en que una unidad ingresa a mantenimiento, y que se cierre automáticamente al normalizarse la condición.

Datos Derivados no Almacenados

Finalidad. Que la longitud de una línea, el porcentaje de ocupación y la diferencia de conciliación coincidan siempre con los datos que los originan.

Fundamento. Un total almacenado requiere actualización explícita. La omisión de esa actualización produce información incorrecta presentada con apariencia de validez.

Problema que resuelve. La existencia de reportes contradictorios entre sí.

Bitácora de Auditoría con Identificación del Usuario Real

Finalidad. Permitir la reconstrucción de qué ocurrió, cuándo y por decisión de quién.

Fundamento. Lo habitual es que la aplicación registre en la bitácora de forma discrecional y que el motor identifique al usuario técnico de la conexión, común a todos. En este diseño, el identificador de la persona se publica en la sesión de base de datos en cada transacción, y un disparador genérico cubre 14 tablas críticas.

Problema que resuelve. Que ante un dato incorrecto se determine quién lo modificó y cuál era su valor anterior. La información del sistema está sujeta al régimen de acceso a la

información pública (Congreso de la República de Guatemala, 2008), por lo que la trazabilidad constituye una obligación legal y no una característica opcional.

Base de Datos Administrada en la Nube

Finalidad. Contar con respaldo automático, réplica y procedimiento de recuperación sin un equipo dedicado de infraestructura.

Fundamento. Un servidor propio exige verificación diaria de los respaldos, actividad que en la práctica se omite hasta que surge la necesidad de restaurar.

Problema que resuelve. Los requerimientos no funcionales de respaldo automático, plan de recuperación ante desastres y disponibilidad igual o mayor al 99 % en horario operativo.

Problemática Resuelta por Requerimiento

Tabla 4

Aporte de cada requerimiento funcional

Requerimiento	Finalidad	Problema manual que elimina
RF-01	Catálogo de líneas y estaciones	Listados paralelos por municipalidad
RF-02	Distancia entre estaciones y longitud de línea	Cálculo manual y totales desactualizados
RF-03	Inventario de accesos	Desconocimiento de la cantidad de guardias requeridos
RF-04	Una unidad en una sola línea activa	Doble asignación y conteo inflado de la flota
RF-05	Alerta por rango de unidades	Detección tardía de la falta de servicio
RF-06	Asignación de parqueos	Unidades sin espacio asignado al final de la jornada
RF-07	Expediente de pilotos	Expedientes en papel y licencias vencidas sin detectar
RF-08	Cobertura de guardias por acceso	Accesos descubiertos detectados fuera de tiempo
RF-09	Ocupación y sobrecupo	Ausencia de evidencia del sobrecupo
RF-10	Regla de espera de cinco minutos	Cumplimiento discrecional de la regla
RF-11	Estado de equipo y conectividad	Mantenimiento reactivo en lugar de planificado
RF-12	Conciliación de pagos	Descuadres detectados cuando ya no son rastreables
RF-13	Asociación a municipalidad	Indefinición de responsabilidades por estación y línea
RF-14	Reportes consolidados	Recolección manual de datos ante cada operador
RF-15	Usuarios y roles	Cuentas compartidas y accesos indebidos
RF-16	Bitácora de auditoría	Imposibilidad de rastrear cambios ante reclamos o auditorías

Nota. Elaboración propia a partir de la especificación de requerimientos del proyecto.

Estado de Avance

Tabla 5

Alcance implementado y pendiente al cierre del Sprint 0

Estado	**Alcance**
Implementado y verificado	Modelo de datos completo de 32 tablas; reglas RF-04, RF-05, RF-06, RF-08, RF-09, RF-10 y RF-16 implementadas en el motor; seguridad y matriz de permisos (RF-15); módulos de líneas, estaciones, accesos, flota y parqueos (RF-01 a RF-06)
Modelado y pendiente de construcción	Personal (RF-07 y RF-08), monitoreo (RF-09 y RF-10), administración técnica (RF-11), pagos (RF-12) y reportería (RF-14); las tablas y reglas ya existen y resta la interfaz y los puntos de acceso
Verificación realizada	Ejecución completa del script en PostgreSQL 18.3 en dos ocasiones consecutivas sin errores, con pruebas de reglas mediante datos válidos e inválidos

Nota. El detalle de la planificación por sprint se encuentra en el documento de plan de desarrollo del proyecto.

Conclusiones

El sistema convierte reglas que actualmente dependen de la verificación humana en restricciones que el motor de base de datos no permite transgredir, y conserva evidencia de cada cambio. Las nueve situaciones problemáticas identificadas en la operación actual quedan cubiertas por los dieciséis requerimientos funcionales, de los cuales siete se encuentran implementados y verificados al cierre del Sprint 0.

La decisión de mayor impacto es la ubicación de las reglas de negocio en el motor de base de datos. Su efecto es verificable: durante las pruebas, los intentos de asignar una unidad a dos líneas, de ocupar un espacio de parqueo ya asignado y de despachar una unidad antes del tiempo mínimo fueron rechazados por el motor, sin intervención del código de la aplicación.

Referencias

Congreso de la República de Guatemala. (2008). *Decreto 57-2008, Ley de Acceso a la Información Pública*. Diario de Centro América.

ISO/IEC. (2011). *ISO/IEC 25010:2011. Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. International Organization for Standardization.

PostgreSQL Global Development Group. (s.f.). *PostgreSQL documentation*. https://www.postgresql.org/docs/

Pressman, R. S., y Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9.ª ed.). McGraw-Hill Education.

Sommerville, I. (2016). *Software engineering* (10.ª ed.). Pearson Education.